

Chapter 12

CS2 course package - Knox College

David P. Bunde¹ and Jaime Spacco²

¹Knox College, dbunde@knox.edu

²Knox College, jspacco@knox.edu

Abstract

At Knox College, CS2 is where we begin having students write actual parallel code. As the programming paradigm, we chose Java's fork-join framework. This fits with the rest of the course both because the course is taught in Java and because it comes with a programming analysis framework in terms of work (the total amount done by a program) and span (the length of the program's critical path) that fits well with asymptotic analysis as used in the rest of the course. In addition to the programming itself, we explore race conditions and dependencies through an introduction to dependency graphs. Currently, this material fits within a couple of lectures, though we plan to move the introduction of parallel computing earlier in the term so that it can be applied to other programming assignments throughout the term.

Descriptor Elements

Textbook Used: None.

Programming Language Employed: Java

Relevant core courses: CS2/Data Structures

Pre-requisites: Students are assumed to have had one previous programming course in Java (CS1). Transfer students sometimes also take the course after taking CS1 in another programming language and this can work provided that they quickly come up to speed on Java. (The early course assignments have an element of review.)

Relevant PDC topics: Classification of PDC Topics according to Bloom's Taxonomy[1]: Remember (RE), Understand (UN), Apply (AP), Analyze (AN), Evaluate (EV), and Create (CR)

PDC Concept	Chapter Section				
	1.1	1.2	1.3	1.4	1.5
Topic1	K	C	A		
Topic2	K	C	A		
Topic3		C	A	A	

Learning outcomes:

1. |Outcome 1|
2. |Outcome 2|

Context for use: Knox College is a small, residential liberal arts institution. It uses an academic calendar with 3 terms (Fall, Winter, and Spring), each slightly less than 10 weeks long. The CS2 class meets 4 times a week, 3 lectures (MWF) and a lab on either Tuesday or Thursday (which depends on the term, but all labs are on the same day each term). All class meetings are 70 minutes long. The intent is that a Knox course should be equivalent to a semester course at other institutions.

The size of this class varies, but the targeted size is 24 students. The class is the second required course in the Computer Science major and is also required for the Computer Science minor and the Data Science major. Historically, roughly half of the students continue taking CS courses after this course and complete one of these. The remaining students use the course as an elective and come from a wide variety of majors.

This course uses Peer Instruction [2, 3], a pedagogy where lecture is periodically interrupted with a multiple-choice question. The students answer these individually, then discuss them in groups and re-answer. Then the instructor reveals the correct answer, has a student explain this answer, and then fills in any additional material. The in-class questions are graded only for participation. Lab assignments aren't graded, but are used as a time for students to practice the material with the instructor and a TA available to answer questions. (Variations of some lab problems are assigned as homework, which is graded.)

12.1 Introduction

Our goal with this course was a general modernization while maintaining the flavor of a data structures course. The biggest change was the introduction of the fork-join framework for writing parallel programs. There was also a slight change of emphasis, however, from implementing the data structures from scratch to using library code more.

12.2 How CS2 was changed and PDC topics integrated

12.2.1 Syllabus

Here is the new course calendar at a week-by-week level:

Week	Topics
1	Course introduction; Java review; ADTs; implementing array-based lists
2	Automated testing; JUnit; writing test cases; OO philosophy; Java interfaces
3	Linked lists; Iterator ADT
4	Running time; Big-O notation; Stack and Queue ADTs and implementations; StringBuilder; design problems
5	Midterm exam ; Sorting algorithms (selection, insertion, bubble, quick); comparators; logarithms; backtracking
6	Map and Set ADTs; binary search; trees; binary search trees; tree traversals
7	Recursion for subset enumeration; recursive linked list operations; parallel computing; beginning of fork-join framework
8	Work and span metrics; Priority Queue ADT; heaps; bucket and radix sorting
9	Hashing; design problem practice; reductions; graphs; graph traversal
10	Review for final

Day-by-day level detail as well as the specific lecture materials and assignments are provided in this chapter's appendix.

12.2.2 Summary of changes

|What remained the same, and what specifically needed to be changed or deleted?|

In addition, the course switched to in-person written exams from take-home exams due to the ability of generative AI systems to solve many of the course's written problems.

12.2.3 Challenges

|Discuss logistical, pedagogical, or technical difficulties.|

12.3 Introducing the Fork-join framework

12.3.1 Problem Description, Prerequisites, and Learning Outcomes

12.3.2 Implementation Details

- Logistics (Material required for the activity, etc.)
- How to Conduct
- Instructional materiel (Lectures, Videos, Assignments, etc.)

12.3.3 Experience and Teaching Tips

12.3.4 Data and Analysis

We did not specifically gather data on this unit of the course. It may be possible, with IRB approval, to analyze student responses to clicker questions to assess their learning of the material on dependency graphs and/or to analyze the code they submitted for the associated programming assignments to assess their ability to use the fork-join framework. Better is probably to do this assessment to a later version of the course in which parallelism is introduced earlier and used more widely in the course.

12.4 Course-wide Pre and Post Course Surveys

We gave pre- and post-surveys for this course during each term that it was offered, but haven't analyzed the data yet.

12.5 Other Activities and Ideas

[Alternative ways to apply these concepts, etc..]

12.6 Conclusion

Bibliography

- [1] Lorin W. Anderson and David R. Krathwohl, editors. A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. Longman, New York, NY, 2001.
- [2] L. Porter, C.B. Lee, and B. Simon. Halving fail rates using peer instruction: a study of four computer science courses. In Proc. 44th SIGCSE Technical Symp. Computer Science Education (SIGCSE TS), pages 177–182, 2013.
- [3] Leo Porter and Beth Simon. Retaining nearly one-third more majors with a trio of instructional best practices in CS1. In Proc. 44th SIGCSE Technical Symp. Computer Science Education (SIGCSE TS), pages 165–170, 2013.

Appendix

Course materials

The materials for this chapter are available at https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS2/chapter3-Knox.

Organization of Material

The directory `ForkJoin` gives an updated version of the materials to teach fork-join programming. It has the slides that I use to teach this material with notes added to indicate what I say on each. It also has a pair of videos of me giving lectures using this material (but using slightly older slides) and three lab assignments that students can use to practice the material. I've used two of these in class; the third is provided as an alternative (the main labs are pretty minimalist, but this one is a more interesting application and requires the students to fill in part of a given program).

To illustrate how this material can be added to a full course, the repo also gives all the lectures and assignments from an offering of the course (from Spring 2025). A syllabus for the course is provided at https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS2/chapter3-Knox/syllabus.pdf and https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS2/chapter3-Knox/schedule.md gives a breakdown of the course by day. The schedule contains links to the materials for each day, serving as a table of contents to organize the other materials, which are stored in the directories `lectures`, `labs`, `exams` and `homework`, which have the actual course materials.

Survey and Other Anonymized Data and Analysis

We gathered student surveys about the course, but haven't processed this data yet.